

Advanced Python & FastAPI

A Working-Code Reference Guide

Interview & Production Reference

September 2026

Part 1: Advanced Python

1. `*args` and `**kwargs`
2. Comprehensions
3. Generators & `yield`
4. Lambda Functions
5. Decorators
6. Context Managers
7. Exception Handling
8. `typing` and `Protocol`
9. Dataclasses
10. `classmethod`, `staticmethod`, `property`
11. Mutable Default Arguments (a classic Python gotcha)
12. Shallow Copy vs. Deep Copy
13. SOLID Principles & Dependency Inversion
14. `async` / `await` and Coroutines
15. `asyncio.gather` and Tasks
16. Concurrency vs. Parallelism

Part 2: Advanced FastAPI

1. Pydantic v2 Fundamentals
2. Request & Response Models
3. Type Hints Drive Everything
4. Async Endpoints
5. Dependency Injection
6. Dependency Injection for Testing
7. Blocking vs. Non-Blocking Work (The #1 FastAPI Production Mistake)
8. Concurrent I/O in Routes with `asyncio.gather`
9. Exception Handling in FastAPI
10. Middleware
11. WebSockets
12. Long-Running AI Workflows
13. File Uploads
14. Authentication vs. Authorization

Quick Reference Cheat Sheet

Advanced Python
Advanced FastAPI

Part 1: Advanced Python

1. `*args` and `**kwargs`

Concept: `*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dict. They let a function accept a flexible, unknown-in-advance number of arguments — the backbone of decorators, wrappers, and generic APIs.

```
def summarize(*args, **kwargs):
    print("positional:", args)    # tuple
    print("keyword:", kwargs)     # dict

summarize(1, 2, 3, name="Alice", age=30)
# positional: (1, 2, 3)
# keyword: {'name': 'Alice', 'age': 30}

# Forwarding args/kwargs to another function (very common in wrappers)
def logged_call(func, *args, **kwargs):
    print(f"Calling {func.__name__} with {args}, {kwargs}")
    return func(*args, **kwargs)

def add(a, b, *, scale=1):
    return (a + b) * scale

result = logged_call(add, 2, 3, scale=10)
print(result) # 50
```

Interview note: The `*` before `scale` in `add` forces `scale` to be keyword-only — a common pattern for readable, misuse-resistant APIs.

2. Comprehensions

Concept: Comprehensions build lists, sets, dicts, or generators in a single, declarative expression. They're faster than manual loops (no repeated `.append()` calls) and considered more "Pythonic," but nested comprehensions hurt readability — a common interview trap.

```
# List comprehension
squares = [x**2 for x in range(10) if x % 2 == 0]
print(squares) # [0, 4, 16, 36, 64]

# Dict comprehension
word_lengths = {w: len(w) for w in ["hi", "hello", "hey"]}
print(word_lengths) # {'hi': 2, 'hello': 5, 'hey': 3}

# Set comprehension
unique_lengths = {len(w) for w in ["hi", "hello", "hey", "yo"]}
print(unique_lengths) # {2, 5}

# Nested comprehension (flatten a matrix) — use sparingly, readability drops fast
matrix = [[1, 2], [3, 4], [5, 6]]
flat = [num for row in matrix for num in row]
print(flat) # [1, 2, 3, 4, 5, 6]

# Generator expression (lazy — no memory blowup, see Section 3)
sum_of_squares = sum(x**2 for x in range(1_000_000))
```

Best practice: If a comprehension needs more than one `if / for` clause or a nested ternary, switch to a regular loop for readability.

3. Generators & `yield`

Concept: A generator function uses `yield` instead of `return` to produce a *stream* of values lazily — one at a time, on demand — instead of building the whole result in memory. This is critical for large datasets, infinite sequences, and pipelines.

```
def fibonacci(limit):
    a, b = 0, 1
    while a < limit:
        yield a          # pauses here, resumes on next()
        a, b = b, a + b

for num in fibonacci(20):
    print(num, end=" ")
# 0 1 1 2 3 5 8 13

# Generators are single-use iterators
gen = fibonacci(10)
print(next(gen)) # 0
print(next(gen)) # 1

# Generator pipeline: lazily process a huge file without loading it all
def read_large_file(path):
    with open(path) as f:
        for line in f:
            yield line.strip()

def filter_errors(lines):
    for line in lines:
        if "ERROR" in line:
            yield line

# Nothing is read into memory until iterated:
# for error_line in filter_errors(read_large_file("app.log")):
#     print(error_line)

# yield from delegates to a sub-generator
def chain_generators(*gens):
    for gen in gens:
        yield from gen

combined = chain_generators(fibonacci(5), fibonacci(3))
print(list(combined)) # [0, 1, 1, 2, 3, 0, 1, 1, 2]
```

Interview note: Generators implement the iterator protocol (`__iter__` / `__next__`) automatically. They trade memory for the constraint that you can only iterate forward, once.

4. Lambda Functions

Concept: `lambda` creates small, anonymous, single-expression functions — useful as throwaway callbacks (sort keys, `map` / `filter` predicates), but should never replace a named `def` for anything non-trivial.

```
add = lambda x, y: x + y
print(add(3, 4)) # 7

# Common use: sort key
people = [{"name": "Bob", "age": 25}, {"name": "Amy", "age": 30}]
people_by_age = sorted(people, key=lambda p: p["age"])
print(people_by_age)

# map / filter
nums = [1, 2, 3, 4, 5]
doubled = list(map(lambda x: x * 2, nums))
evens = list(filter(lambda x: x % 2 == 0, nums))
print(doubled, evens) # [2, 4, 6, 8, 10] [2, 4]
```

Pitfall: Lambdas can't contain statements (no `if/else` blocks, no assignments beyond a conditional expression), and stack traces referencing `<lambda>` are hard to debug — prefer `def` for anything reused or complex.

5. Decorators

Concept: A decorator is a function that wraps another function to extend its behavior without modifying its source — the mechanism behind `@staticmethod`, `@property`, FastAPI's `@app.get(...)`, caching, logging, timing, and auth checks.

```

import functools
import time

def timer(func):
    @functools.wraps(func)  # preserves func.__name__, docstring, etc.
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} took {elapsed:.4f}s")
        return result
    return wrapper

@timer
def slow_square(n):
    time.sleep(0.1)
    return n ** 2

slow_square(5)
# slow_square took 0.1001s

# Decorator with arguments (a decorator factory)
def retry(times=3):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    print(f"Attempt {attempt} failed: {e}")
                    if attempt == times:
                        raise
            return wrapper
        return decorator

@retry(times=3)
def flaky():
    import random
    if random.random() < 0.7:
        raise ValueError("Random failure")
    return "Success"

# flaky() # Retries up to 3 times before propagating the error

```

Why `functools.wraps` matters: Without it, `slow_square.__name__` would be `"wrapper"`, breaking introspection, logging, and tools (like FastAPI) that inspect function metadata.

6. Context Managers

Concept: Context managers guarantee setup/teardown code runs, even on exceptions — the `with` statement's `__enter__` / `__exit__` protocol. Used for files, DB connections, locks, and timing blocks.

```

# Class-based context manager
class DatabaseConnection:
    def __enter__(self):
        print("Opening connection")
        self.conn = "fake_connection_object"
        return self.conn

    def __exit__(self, exc_type, exc_value, traceback):
        print("Closing connection")
        # Return True to suppress the exception; False/None lets it propagate
        if exc_type is not None:
            print(f"Handled error: {exc_value}")
        return False

with DatabaseConnection() as conn:
    print(f"Using {conn}")
# Opening connection
# Using fake_connection_object
# Closing connection

# Function-based context manager (contextlib) – simpler for one-off cases
from contextlib import contextmanager

@contextmanager
def timer_block(label):
    start = time.perf_counter()
    try:
        yield # code inside `with` runs here
    finally:
        print(f"{label}: {time.perf_counter() - start:.4f}s")

with timer_block("expensive operation"):
    time.sleep(0.05)
# expensive operation: 0.0501s

```

Interview note: `__exit__` receiving exception info is what lets context managers implement cleanup-and-suppress or cleanup-and-raise logic — the same pattern powers FastAPI’s dependency `yield` cleanup (see Part 2 §5).

7. Exception Handling

Concept: Python exceptions form a class hierarchy; catch the most specific exception you can handle, use `else` for code that should run only if no exception occurred, and `finally` for guaranteed cleanup.

```

class InsufficientFundsError(Exception):
    """Custom exception with extra context."""
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(f"Cannot withdraw {amount}, balance is {balance}")

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(balance, amount)
    return balance - amount

try:
    new_balance = withdraw(100, 150)
except InsufficientFundsError as e:
    print(f"Error: {e}, shortfall: {e.amount - e.balance}")
except (ValueError, TypeError) as e:
    print(f"Bad input: {e}")
else:
    print(f"Success, new balance: {new_balance}") # only if no exception
finally:
    print("Transaction attempt logged") # always runs

# Exception chaining – preserves the original cause for debugging
def load_config():
    try:
        raise FileNotFoundError("config.yaml missing")
    except FileNotFoundError as e:
        raise RuntimeError("Failed to start app") from e

```

Best practice: Never use a bare `except:` — it silently swallows `KeyboardInterrupt` and `SystemExit` too. Catch specific exception types.

8. typing and Protocol

Concept: Type hints don't change runtime behavior but power static analysis (mypy), IDE autocomplete, and — critically for FastAPI — automatic request validation. `Protocol` enables *structural* typing ("duck typing" with type-checker support): a class satisfies a Protocol simply by having the right methods, no inheritance required.

```

from typing import Optional, Union, TypedDict, Protocol, runtime_checkable

# Standard hints
def greet(name: str, times: int = 1) -> list[str]:
    return [f"Hello, {name}"] * times

# Optional == Union[X, None]
def find_user(user_id: int) -> Optional[dict]:
    return None # not found

# Union for multiple accepted types
def process(value: Union[int, str]) -> str:
    return str(value)

# TypedDict — a dict with a known, checked shape
class UserDict(TypedDict):
    id: int
    name: str
    active: bool

user: UserDict = {"id": 1, "name": "Alice", "active": True}

# Protocol — structural typing
@runtime_checkable
class Serializable(Protocol):
    def to_dict(self) -> dict: ...

class Product:
    def __init__(self, name: str, price: float):
        self.name, self.price = name, price

    def to_dict(self) -> dict:
        return {"name": self.name, "price": self.price}

def save(obj: Serializable) -> None:
    print(f"Saving {obj.to_dict()}")

save(Product("Widget", 9.99)) # works — no explicit inheritance needed!
print(isinstance(Product("x", 1), Serializable)) # True, thanks to @runtime_checkable

```

Interview note: `Protocol` is Python's answer to interfaces (like Go or TypeScript). It decouples code from concrete base classes — a key tool for dependency inversion (see §12).

9. Dataclasses

Concept: `@dataclass` auto-generates `__init__`, `__repr__`, and `__eq__` from class-level type annotations, eliminating boilerplate for simple data-holding classes.

```

from dataclasses import dataclass, field

@dataclass
class Point:
    x: float
    y: float
    label: str = "unlabeled" # default value

p1 = Point(1.0, 2.0)
p2 = Point(1.0, 2.0)
print(p1)          # Point(x=1.0, y=2.0, label='unlabeled')
print(p1 == p2)     # True - __eq__ generated automatically

@dataclass(frozen=True) # immutable - hashable, raises on mutation attempts
class ImmutablePoint:
    x: float
    y: float

ip = ImmutablePoint(1, 2)
# ip.x = 5 # raises dataclasses.FrozenInstanceError

@dataclass
class Team:
    name: str
    # Mutable defaults MUST use field(default_factory=...) - see Section 10
    members: list[str] = field(default_factory=list)

    def __post_init__(self):
        # Runs right after __init__ - good for validation/derived fields
        if not self.name:
            raise ValueError("Team name required")

team = Team("Backend")
team.members.append("Alice")
print(team) # Team(name='Backend', members=['Alice'])

```

Interview note: `@dataclass` is the standard-library ancestor of Pydantic's `BaseModel` — dataclasses give you structure and boilerplate reduction; Pydantic (Part 2) adds runtime *validation and parsing*, which is why FastAPI uses Pydantic instead.

10. `classmethod`, `staticmethod`, `property`

Concept: `@classmethod` receives the class (`cls`) instead of an instance — used for alternative constructors. `@staticmethod` receives neither `self` nor `cls` — a namespaced plain function. `@property` turns a method into an attribute-like accessor, enabling computed fields and validation on assignment.

```

class Temperature:
    def __init__(self, celsius: float):
        self._celsius = celsius

    @property
    def celsius(self) -> float:
        return self._celsius

    @celsius.setter
    def celsius(self, value: float):
        if value < -273.15:
            raise ValueError("Below absolute zero")
        self._celsius = value

    @property
    def fahrenheit(self) -> float:
        return self._celsius * 9 / 5 + 32 # computed, read-only

    @classmethod
    def from_fahrenheit(cls, f: float) -> "Temperature":
        return cls((f - 32) * 5 / 9) # alternative constructor

    @staticmethod
    def is_valid(celsius: float) -> bool:
        return celsius >= -273.15 # pure utility, no instance/class needed

t = Temperature(25)
print(t.fahrenheit) # 77.0 (looks like an attribute, is a method)
t.celsius = 30 # validated via setter
print(Temperature.is_valid(-500)) # False

t2 = Temperature.from_fahrenheit(98.6)
print(round(t2.celsius, 1)) # 37.0

```

When to use which: - `@property` — expose computed or validated attributes with clean syntax. - `@classmethod` — alternative constructors, factory methods (`.from_json()` , `.from_fahrenheit()`). - `@staticmethod` — a helper that's logically grouped with the class but needs no instance/class state.

11. Mutable Default Arguments (a classic Python gotcha)

Concept: Default argument values are evaluated **once**, when the function is defined — not on every call. If that default is mutable (list, dict, set), every call that doesn't override it *shares the same object*, causing surprising, hard-to-debug state leakage.

```

# THE BUG
def add_item(item, basket=[]):    # basket created ONCE at def time
    basket.append(item)
    return basket

print(add_item("apple"))    # ['apple']
print(add_item("banana"))  # ['apple', 'banana'] <-- unexpected! same list reused

# THE FIX — use None as sentinel, create fresh default inside the function
def add_item_fixed(item, basket=None):
    if basket is None:
        basket = []
    basket.append(item)
    return basket

print(add_item_fixed("apple"))    # ['apple']
print(add_item_fixed("banana"))  # ['banana'] <-- correct, independent lists

```

Interview note: This is one of the most common Python interview “gotcha” questions. The same issue applies to dataclass fields (`field(default_factory=list)`) exists specifically to solve it — see §9).

12. Shallow Copy vs. Deep Copy

Concept: A shallow copy duplicates the outer container but keeps references to the same nested objects. A deep copy recursively duplicates everything, so nested mutations don’t leak between the original and the copy.

```

import copy

original = {"name": "Alice", "scores": [90, 85, 95]}

shallow = original.copy()    # or copy.copy(original)
deep = copy.deepcopy(original)

shallow["scores"].append(100)    # mutates the SHARED inner list
print(original["scores"])        # [90, 85, 95, 100] <-- original changed too!
print(deep["scores"])            # [90, 85, 95] <-- unaffected

shallow["name"] = "Bob"          # reassigning a top-level key is independent
print(original["name"])          # 'Alice' — unaffected, because this creates
                                # a new binding at the top level, not a mutation

# Same principle with lists of objects
class Config:
    def __init__(self, settings):
        self.settings = settings

c1 = Config({"debug": True})
c2 = copy.copy(c1)              # shallow: c2.settings IS c1.settings (same dict)
c3 = copy.deepcopy(c1)          # deep: c3.settings is an independent dict

c2.settings["debug"] = False
print(c1.settings) # {'debug': False} <-- shared reference mutated
print(c3.settings) # {'debug': True} <-- fully independent

```

Rule of thumb: Shallow copy is fine (and faster) for flat structures or when you intentionally want shared references. Use `deepcopy` whenever nested mutable objects must be fully independent — e.g., cloning a config object you’re about to modify per-request.

13. SOLID Principles & Dependency Inversion

Concept: SOLID is five object-oriented design principles that keep code maintainable as it grows. The most interview-relevant for backend/API work is the last one — **Dependency Inversion** — because it's the theoretical basis for FastAPI's entire dependency injection system (Part 2 §5).

```
from abc import ABC, abstractmethod
from typing import Protocol

# S – Single Responsibility: a class should have one reason to change.
class InvoiceCalculator:
    def calculate_total(self, items: list[float]) -> float:
        return sum(items)

class InvoicePrinter:
    # printing is a SEPARATE responsibility
    def print(self, total: float) -> None:
        print(f"Total: ${total:.2f}")

# O – Open/Closed: open for extension, closed for modification.
class DiscountStrategy(ABC):
    @abstractmethod
    def apply(self, total: float) -> float: ...

class NoDiscount(DiscountStrategy):
    def apply(self, total: float) -> float:
        return total

class PercentageDiscount(DiscountStrategy):
    def __init__(self, percent: float):
        self.percent = percent

    def apply(self, total: float) -> float:
        return total * (1 - self.percent / 100)

# Adding a new discount type means a NEW class, no changes to existing code.

# L – Liskov Substitution: subclasses must be usable wherever the base is expected.
def checkout(total: float, discount: DiscountStrategy) -> float:
    return discount.apply(total) # works identically for ANY DiscountStrategy

# I – Interface Segregation: prefer small, focused interfaces over one giant one.
class Readable(Protocol):
    def read(self) -> str: ...

class Writable(Protocol):
    def write(self, data: str) -> None: ...

# A read-only client only needs to depend on Readable, not a bloated ReadWrite interface.

# D – Dependency Inversion: depend on abstractions, not concrete implementations.
class NotificationSender(Protocol):
    def send(self, message: str) -> None: ...

class EmailSender:
    def send(self, message: str) -> None:
        print(f"Emailing: {message}")

class SMSSender:
```

```

def send(self, message: str) -> None:
    print(f"Texting: {message}")

class OrderService:
    # Depends on the ABSTRACTION (NotificationSender), not a concrete class.
    def __init__(self, sender: NotificationSender):
        self.sender = sender

    def place_order(self, item: str):
        print(f"Order placed: {item}")
        self.sender.send(f"Your order for {item} is confirmed")

    # The concrete implementation is "injected" from outside – swap freely, and
    # in tests, inject a fake sender instead of a real EmailSender.
    service = OrderService(sender=EmailSender())
    service.place_order("Laptop")

    service_sms = OrderService(sender=SMSSender())
    service_sms.place_order("Phone")

```

Why this matters for FastAPI: `OrderService(sender=...)` — passing a dependency in from outside rather than constructing it internally — is exactly the pattern FastAPI's `Depends()` automates. Understanding DI here makes FastAPI's dependency system feel obvious rather than magical.

14. `async` / `await` and Coroutines

Concept: `async def` defines a *coroutine function*. Calling it doesn't run the code — it returns a coroutine object that must be **awaited** (or scheduled) to actually execute. `await` suspends the current coroutine at an I/O boundary, letting the event loop run other work in the meantime — this is cooperative, single-threaded concurrency, not parallelism.

```

import asyncio

async def fetch_data(name: str, delay: float) -> str:
    print(f"{name}: starting")
    await asyncio.sleep(delay) # simulates non-blocking I/O (e.g. a network call)
    print(f"{name}: done")
    return f"{name} result"

async def main():
    # Awaiting sequentially: total time = sum of delays
    result = await fetch_data("A", 1)
    print(result)

asyncio.run(main())

```

Common misconception: `async def foo(): ...; foo()` does *not* run the function — it silently creates a coroutine object that's never awaited (Python even warns about this). You must `await foo()` or schedule it as a Task.

15. `asyncio.gather` and Tasks

Concept: Awaiting coroutines one at a time is still sequential — you get no concurrency benefit. `asyncio.gather()` runs multiple coroutines *concurrently* on the single-threaded event loop, and `asyncio.create_task()` schedules a coroutine to start running in the background immediately (not just when awaited).

```

import asyncio
import time

async def fetch_data(name: str, delay: float) -> str:
    await asyncio.sleep(delay)
    return f"{name} result"

async def sequential_demo():
    start = time.perf_counter()
    r1 = await fetch_data("A", 1)
    r2 = await fetch_data("B", 1)
    print(f"Sequential: {time.perf_counter() - start:.2f}s") # ~2.0s

async def concurrent_demo():
    start = time.perf_counter()
    # Both coroutines run concurrently – total time ≈ max(delays), not sum
    r1, r2 = await asyncio.gather(
        fetch_data("A", 1),
        fetch_data("B", 1),
    )
    print(f"Concurrent: {time.perf_counter() - start:.2f}s") # ~1.0s

async def task_demo():
    # create_task schedules it to start RIGHT NOW, running in the background
    task = asyncio.create_task(fetch_data("Background", 2))
    print("Doing other work while task runs...")
    await asyncio.sleep(0.5)
    print("Other work done, now waiting for the task")
    result = await task # only NOW do we block until it finishes
    print(result)

async def gather_with_errors():
    async def might_fail(n):
        if n == 2:
            raise ValueError(f"Item {n} failed")
        return n * 10

    # return_exceptions=True: failures become values instead of aborting gather()
    results = await asyncio.gather(
        might_fail(1), might_fail(2), might_fail(3),
        return_exceptions=True,
    )
    print(results) # [10, ValueError('Item 2 failed'), 30]

async def main():
    await sequential_demo()
    await concurrent_demo()
    await task_demo()
    await gather_with_errors()

asyncio.run(main())

```

Interview note: Without `return_exceptions=True`, the *first* exception raised inside `gather()` propagates immediately, and the other coroutines' results (though they keep running) are effectively lost to the caller — a common production bug.

16. Concurrency vs. Parallelism

Concept: These are frequently conflated but are fundamentally different strategies for doing more than one thing “at once”:

	Concurrency	Parallelism
Definition	Interleaving tasks on ~1 core; progress on multiple tasks over time	Simultaneous execution on multiple CPU cores
Python tool	<code>asyncio</code> (single-threaded event loop)	<code>multiprocessing</code> (separate processes, real parallel CPU)
Best for	I/O-bound work (network, disk, DB)	CPU-bound work (heavy computation)
GIL impact	Not blocked by GIL (I/O releases it)	Bypasses the GIL entirely (separate processes)

```

import asyncio
import time
from concurrent.futures import ProcessPoolExecutor

# --- Concurrency: great for I/O-bound work ---
async def io_bound_task(n):
    await asyncio.sleep(1)  # simulates waiting on network/disk - doesn't burn CPU
    return n

async def concurrency_demo():
    start = time.perf_counter()
    await asyncio.gather(*(io_bound_task(i) for i in range(5)))
    print(f"5 concurrent I/O tasks: {time.perf_counter() - start:.2f}s") # ~1s, not 5s

# --- Parallelism: needed for CPU-bound work ---
def cpu_bound_task(n):
    total = 0
    for i in range(10_000_000): # real CPU work - asyncio would NOT speed this up
        total += i
    return total

def parallelism_demo():
    start = time.perf_counter()
    with ProcessPoolExecutor(max_workers=4) as executor:
        results = list(executor.map(cpu_bound_task, range(4)))
    print(f"4 parallel CPU tasks: {time.perf_counter() - start:.2f}s")

asyncio.run(concurrency_demo())
# parallelism_demo() # run in __main__ guard on real systems

```

Interview note — the trap question: “Does `asyncio` make Python multi-threaded?” No. `asyncio` is concurrency on a *single* thread via an event loop; it helps I/O-bound code (many waiting network calls) but does nothing for CPU-bound code, which needs real parallelism (`multiprocessing`) to bypass the GIL. This distinction is exactly why FastAPI treats blocking vs. non-blocking work differently (see Part 2 §6).

Part 2: Advanced FastAPI

This part assumes FastAPI + Pydantic v2 are installed: `pip install "fastapi[standard]"`. Code samples are self-contained apps you can save as `main.py` and run with `fastapi dev main.py` or `uvicorn main:app --reload`.

1. Pydantic v2 Fundamentals

Concept: Pydantic models are the runtime-validating cousin of dataclasses (Part 1 §9). FastAPI uses them to parse incoming JSON, validate types/constraints, and serialize responses — all automatically, from type hints.

```
from pydantic import BaseModel, Field, EmailStr, field_validator, ConfigDict
from datetime import datetime
from typing import Optional

class User(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True) # v2 config style

    id: int
    name: str = Field(min_length=1, max_length=50)
    email: EmailStr
    age: Optional[int] = Field(default=None, ge=0, le=150)
    created_at: datetime = Field(default_factory=datetime.utcnow)

    @field_validator("name")
    @classmethod
    def name_must_be_capitalized(cls, v: str) -> str:
        if not v[0].isupper():
            raise ValueError("Name must start with a capital letter")
        return v

# Parsing & validation happen automatically at construction time
user = User(id=1, name="Alice", email="alice@example.com", age=30)
print(user.model_dump()) # dict representation (v2 name for .dict())
print(user.model_dump_json()) # JSON string

# Validation errors are structured and machine-readable
try:
    User(id=1, name="bob", email="not-an-email")
except Exception as e:
    print(e) # lists BOTH the capitalization and email errors
```

v1 → v2 naming changes to know for interviews: `.dict()` → `.model_dump()`, `.json()` → `.model_dump_json()`, `class Config` → `model_config = ConfigDict(...)`, `@validator` → `@field_validator`. Pydantic v2 is also a Rust-backed core (`pydantic-core`), giving a large speedup over v1.

2. Request & Response Models

Concept: Separate Pydantic models for input and output prevent over-fetching/over-posting bugs (e.g., leaking a password hash) and let FastAPI auto-generate accurate OpenAPI docs.

```

from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

# Input model – what the CLIENT sends
class UserCreate(BaseModel):
    name: str
    email: EmailStr
    password: str

# Output model – what the CLIENT receives (note: no password!)
class UserOut(BaseModel):
    id: int
    name: str
    email: EmailStr

_fake_db: dict[int, dict] = {}
_next_id = 1

@app.post("/users", response_model=UserOut, status_code=201)
def create_user(payload: UserCreate):
    global _next_id
    user = {"id": _next_id, "name": payload.name, "email": payload.email,
           "password_hash": f"hashed({payload.password})"}
    _fake_db[_next_id] = user
    _next_id += 1
    return user  # FastAPI filters this dict down to UserOut's fields automatically

@app.get("/users/{user_id}", response_model=UserOut)
def get_user(user_id: int):
    return _fake_db[user_id]

```

Key point: `response_model=UserOut` means even though the returned dict contains `password_hash`, the client never sees it — FastAPI serializes strictly through the response model.

3. Type Hints Drive Everything

Concept: FastAPI reads Python type hints to know where each piece of input comes from (path, query, body) and how to validate/convert it — there's no separate schema definition layer.

```

from fastapi import FastAPI, Query, Path
from typing import Optional

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(
    item_id: int = Path(..., description="The item's unique ID", ge=1),
    q: Optional[str] = Query(default=None, max_length=50),
    limit: int = Query(default=10, le=100),
):
    # item_id: comes from the URL path, must be int, validated >= 1
    # q: optional query param, e.g. /items/5?q=search
    # limit: query param with a default and max value
    return {"item_id": item_id, "q": q, "limit": limit}

# Request: GET /items/5?q=hello&limit=20
# Response: {"item_id": 5, "q": "hello", "limit": 20}

```

Interview note: The rule of thumb — path/query params come from simple types (`int` , `str` , `bool`) with `Path()` / `Query()` ; a Pydantic `BaseModel` type hint always means “parse this from the JSON request body.”

4. Async Endpoints

Concept: FastAPI supports both `def` and `async def` route handlers. Use `async def` when the handler awaits truly non-blocking I/O (async DB drivers, `httpx.AsyncClient` , etc.) — see §6 for what happens if you get this wrong.

```

from fastapi import FastAPI
import httpx
import asyncio

app = FastAPI()

@app.get("/sync-hello")
def sync_hello():
    return {"message": "Hello (sync)"}

@app.get("/async-hello")
async def async_hello():
    await asyncio.sleep(0.1) # non-blocking simulated I/O
    return {"message": "Hello (async)"}

@app.get("/external-data")
async def get_external_data():
    async with httpx.AsyncClient() as client:
        response = await client.get("https://api.github.com/repos/python/cpython")
        return {"stars": response.json().get("stargazers_count")}

```

5. Dependency Injection

Concept: `Depends()` is FastAPI’s implementation of dependency inversion (Part 1 §13) — instead of a route hard-coding how it gets a DB session, current user, or config, it declares that dependency and FastAPI resolves it, calling shared setup/teardown logic and caching results per-request.

```

from fastapi import FastAPI, Depends, HTTPException, Header
from typing import Annotated

```

```

app = FastAPI()

# --- Simple dependency: shared query params ---
def pagination_params(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}

@app.get("/items")
def list_items(pagination: Annotated[dict, Depends(pagination_params)]):
    return {"pagination": pagination, "items": []}

# --- Dependency with yield: setup + guaranteed teardown (like a context manager) ---
class FakeDBSession:
    def __init__(self):
        print("DB session opened")
    def close(self):
        print("DB session closed")

def get_db():
    db = FakeDBSession()
    try:
        yield db          # everything before yield = setup, after = teardown
    finally:
        db.close()        # ALWAYS runs, even if the route raises an exception

@app.get("/products/{product_id}")
def get_product(product_id: int, db: Annotated[FakeDBSession, Depends(get_db)]):
    return {"product_id": product_id, "db": str(db)}

# --- Chained (nested) dependencies ---
def get_token_header(authorization: Annotated[str | None, Header()] = None):
    if authorization is None:
        raise HTTPException(status_code=401, detail="Missing Authorization header")
    return authorization.replace("Bearer ", "")

def get_current_user(token: Annotated[str, Depends(get_token_header)]):
    # In real code: decode/validate the JWT, look up the user
    if token != "valid-token":
        raise HTTPException(status_code=401, detail="Invalid token")
    return {"username": "alice"}

@app.get("/me")
def read_current_user(user: Annotated[dict, Depends(get_current_user)]):
    return user

# get_current_user depends on get_token_header – FastAPI resolves the whole
# chain automatically and caches each dependency's result within one request.

# --- Class-based dependency (bundles multiple params) ---
class CommonQueryParams:
    def __init__(self, q: str | None = None, page: int = 1, size: int = 20):
        self.q = q
        self.page = page
        self.size = size

@app.get("/search")
def search(params: Annotated[CommonQueryParams, Depends()]):
    return {"q": params.q, "page": params.page, "size": params.size}

```

Why `yield` dependencies matter in production: The `finally` block runs even if the route handler raises an exception — this is how you guarantee a DB connection or file handle is *always* released, mirroring the context manager protocol from Part 1 §6.

6. Dependency Injection for Testing

Concept: Because dependencies are just callables referenced by `Depends()`, FastAPI lets you **override** them in tests via `app.dependency_overrides` — swapping a real database for a fake one, or a real auth check for an always-pass stub, with zero changes to route code.

```
from fastapi import FastAPI, Depends
from fastapi.testclient import TestClient
from typing import Annotated

app = FastAPI()

class RealDatabase:
    def get_user(self, user_id: int):
        # Pretend this hits a real, slow, external database
        raise RuntimeError("Real DB not available in tests!")

def get_db():
    return RealDatabase()

@app.get("/users/{user_id}")
def read_user(user_id: int, db: Annotated[RealDatabase, Depends(get_db)]):
    return db.get_user(user_id)

# --- In test code ---
class FakeDatabase:
    def get_user(self, user_id: int):
        return {"id": user_id, "name": "Test User"}

def override_get_db():
    return FakeDatabase()

# Swap the dependency BEFORE creating the TestClient
app.dependency_overrides[get_db] = override_get_db

client = TestClient(app)

def test_read_user():
    response = client.get("/users/1")
    assert response.status_code == 200
    assert response.json() == {"id": 1, "name": "Test User"}

test_read_user()
print("Test passed!")

# Clean up after tests so overrides don't leak between test modules
app.dependency_overrides.clear()
```

Interview note: This pattern is *only possible* because routes depend on abstractions (`Depends(get_db)`), not concrete classes constructed inline — the exact Dependency Inversion principle from Part 1 §13, applied.

7. Blocking vs. Non-Blocking Work (The #1 FastAPI Production Mistake)

Concept: `async def` route handlers run on the *single-threaded* event loop (Part 1 §14–16). If you call a **blocking** (synchronous, CPU-bound or blocking-I/O) function directly inside an `async def` route, you freeze the entire event loop — **every other concurrent request stalls** until that call finishes.

```
from fastapi import FastAPI
import time
import asyncio

app = FastAPI()

# --- THE BUG: blocking call inside an async route ---
@app.get("/bad")
async def bad_endpoint():
    time.sleep(3)  # BLOCKING — freezes the ENTIRE event loop for 3s.
    return {"status": "done"}  # No other request can be served during this!

# --- FIX 1: if the work is truly synchronous/CPU-bound, use a plain `def` route.
#           FastAPI automatically runs `def` routes in a background thread pool,
#           so they never block the event loop. ---
@app.get("/good-sync")
def good_sync_endpoint():
    time.sleep(3)  # Still "blocking," but it's on a worker thread, not the loop
    return {"status": "done"}

# --- FIX 2: if you're inside an async route but must call blocking code,
#           explicitly offload it to a thread pool. ---
@app.get("/good-async-offload")
async def good_async_offload():
    result = await asyncio.to_thread(time.sleep, 3)
    return {"status": "done"}

# --- FIX 3 (best): use a genuinely async-native library so nothing blocks. ---
@app.get("/good-native-async")
async def good_native_async():
    await asyncio.sleep(3)  # non-blocking; event loop serves other requests meanwhile
    return {"status": "done"}
```

The decision rule (memorize this for interviews): - Route does mostly **I/O** with an **async-native library** (httpx.AsyncClient, asyncpg, motor) → `async def`. - Route does **CPU-bound work** or uses a **sync-only library** (requests, most ORMs by default, PIL/numpy-heavy work) → plain `def` (FastAPI's thread pool protects the loop) or offload via `asyncio.to_thread` / `run_in_executor`. - Never call a blocking function directly inside `async def` — that's the single most common FastAPI production bug.

8. Concurrent I/O in Routes with `asyncio.gather`

Concept: Just like in plain Python (Part 1 §15), a route that needs data from multiple independent async sources should fetch them concurrently, not sequentially — turning `sum(latencies)` into `max(latencies)`.

```

from fastapi import FastAPI
import asyncio
import httpx

app = FastAPI()

async def fetch_weather(city: str) -> dict:
    await asyncio.sleep(0.5) # simulate external API call
    return {"city": city, "temp_c": 22}

async def fetch_traffic(city: str) -> dict:
    await asyncio.sleep(0.5) # simulate another external API call
    return {"city": city, "congestion": "moderate"}

async def fetch_news(city: str) -> dict:
    await asyncio.sleep(0.5)
    return {"city": city, "headline": "Local event today"}

@app.get("/dashboard/{city}")
async def get_dashboard(city: str):
    # Sequential would take ~1.5s total. Concurrent takes ~0.5s (max of the three).
    weather, traffic, news = await asyncio.gather(
        fetch_weather(city),
        fetch_traffic(city),
        fetch_news(city),
    )
    return {"weather": weather, "traffic": traffic, "news": news}

@app.get("/dashboard-resilient/{city}")
async def get_dashboard_resilient(city: str):
    # If one upstream service might fail, don't let it take down the whole response
    results = await asyncio.gather(
        fetch_weather(city), fetch_traffic(city), fetch_news(city),
        return_exceptions=True,
    )
    keys = ["weather", "traffic", "news"]
    return {
        key: (result if not isinstance(result, Exception) else {"error": str(result)})
        for key, result in zip(keys, results)
    }

```

9. Exception Handling in FastAPI

Concept: FastAPI provides `HTTPException` for expected, client-facing errors, and a global `exception_handler` registry for turning unhandled/custom exceptions into consistent JSON error responses across the whole app.

```

from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError

app = FastAPI()

_items = {1: "Widget"}

@app.get("/items/{item_id}")
def read_item(item_id: int):
    if item_id not in _items:
        # HTTPException -> a clean, structured 404 with a detail message
        raise HTTPException(status_code=404, detail=f"Item {item_id} not found")
    return {"item": _items[item_id]}

# --- Custom domain exception ---
class InsufficientStockError(Exception):
    def __init__(self, item: str, requested: int, available: int):
        self.item, self.requested, self.available = item, requested, available

@app.exception_handler(InsufficientStockError)
async def stock_error_handler(request: Request, exc: InsufficientStockError):
    return JSONResponse(
        status_code=409,
        content={
            "error": "insufficient_stock",
            "item": exc.item,
            "requested": exc.requested,
            "available": exc.available,
        },
    )

@app.post("/orders")
def create_order(item: str, quantity: int):
    available = 5
    if quantity > available:
        raise InsufficientStockError(item, quantity, available)
    return {"status": "ordered"}

# --- Override the default 422 validation error shape ---
@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc: RequestValidationError):
    return JSONResponse(
        status_code=422,
        content={"error": "validation_failed", "details": exc.errors()},
    )

# --- Catch-all for truly unexpected errors (never leak internals to clients) ---
@app.exception_handler(Exception)
async def unhandled_exception_handler(request: Request, exc: Exception):
    # Log the real exception server-side here (with traceback) before responding
    return JSONResponse(
        status_code=500,
        content={"error": "internal_server_error"},
    )

```

Best practice: Never let a raw exception's message/traceback reach the client in production — the catch-all handler above logs internally but returns a generic message, preventing information leakage.

10. Middleware

Concept: Middleware wraps every request/response — the ASGI-level equivalent of a decorator applied globally. Used for logging, timing, CORS, auth headers, and request IDs.

```
from fastapi import FastAPI, Request
from starlette.middleware.base import BaseHTTPMiddleware
from fastapi.middleware.cors import CORSMiddleware
import time
import uuid

app = FastAPI()

# --- Built-in middleware: CORS ---
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://myfrontend.com"],
    allow_methods=["GET", "POST"],
    allow_headers=["*"],
)

# --- Custom middleware: request timing + request ID ---
class TimingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        request_id = str(uuid.uuid4())
        start = time.perf_counter()

        response = await call_next(request)  # runs the route (and any inner middleware)

        elapsed = time.perf_counter() - start
        response.headers["X-Request-ID"] = request_id
        response.headers["X-Process-Time"] = f"{elapsed:.4f}"
        print(f"[{request_id}] {request.method} {request.url.path} - {elapsed:.4f}s")
        return response

app.add_middleware(TimingMiddleware)

# --- Function-style middleware (simpler for one-off cases) ---
@app.middleware("http")
async def add_security_headers(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Content-Type-Options"] = "nosniff"
    return response

@app.get("/ping")
def ping():
    return {"status": "ok"}
```

Execution order: Middleware added last runs *first* on the way in (and last on the way out) — they nest like an onion around the route handler.

11. WebSockets

Concept: WebSockets provide a persistent, full-duplex connection — essential for chat apps, live dashboards, and streaming AI responses token-by-token (as opposed to request/response HTTP).

```

from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from typing import List

app = FastAPI()

class ConnectionManager:
    def __init__(self):
        self.active_connections: List[WebSocket] = []

    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)

    def disconnect(self, websocket: WebSocket):
        self.active_connections.remove(websocket)

    async def broadcast(self, message: str):
        for connection in self.active_connections:
            await connection.send_text(message)

manager = ConnectionManager()

@app.websocket("/ws/chat/{client_id}")
async def chat_endpoint(websocket: WebSocket, client_id: str):
    await manager.connect(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            await manager.broadcast(f"Client #{client_id}: {data}")
    except WebSocketDisconnect:
        manager.disconnect(websocket)
        await manager.broadcast(f"Client #{client_id} left the chat")

# --- Streaming token-by-token, e.g. an LLM response (see Section 12) ---
@app.websocket("/ws/stream")
async def stream_endpoint(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            prompt = await websocket.receive_text()
            fake_tokens = ["The", " answer", " is", " 42", "."]
            for token in fake_tokens:
                await websocket.send_text(token)
                import asyncio
                await asyncio.sleep(0.1) # simulate generation delay
            await websocket.send_json({"event": "done"})
    except WebSocketDisconnect:
        print("Client disconnected")

```

12. Long-Running AI Workflows

Concept: LLM calls and agent workflows (see the companion LangChain/LangGraph guides) can take seconds to minutes. Three production-proven patterns: (a) stream partial results as they're generated, (b) run the work in the background and let the client poll, or (c) push a webhook when done.

```

from fastapi import FastAPI, BackgroundTasks, HTTPException
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
import asyncio
import uuid

app = FastAPI()

# --- Pattern A: Server-Sent Events / streaming response ---
async def generate_llm_tokens(prompt: str):
    fake_response = "This is a simulated streaming LLM response."
    for word in fake_response.split():
        await asyncio.sleep(0.15) # simulate token generation latency
        yield f"{word} "

@app.get("/generate-stream")
async def generate_stream(prompt: str):
    return StreamingResponse(generate_llm_tokens(prompt), media_type="text/plain")

# --- Pattern B: background task + polling (for multi-second/minute workflows) ---
class JobStatus(BaseModel):
    job_id: str
    status: str # "pending" | "running" | "complete" | "failed"
    result: str | None = None

_jobs: dict[str, JobStatus] = {}

async def run_agent_workflow(job_id: str, task: str):
    _jobs[job_id].status = "running"
    try:
        await asyncio.sleep(5) # simulate a slow multi-step agent workflow
        _jobs[job_id].status = "complete"
        _jobs[job_id].result = f"Completed analysis of: {task}"
    except Exception as e:
        _jobs[job_id].status = "failed"
        _jobs[job_id].result = str(e)

@app.post("/jobs", response_model=JobStatus, status_code=202)
async def start_job(task: str, background_tasks: BackgroundTasks):
    job_id = str(uuid.uuid4())
    _jobs[job_id] = JobStatus(job_id=job_id, status="pending")
    background_tasks.add_task(run_agent_workflow, job_id, task)
    return _jobs[job_id] # returns immediately - 202 Accepted, work continues async

@app.get("/jobs/{job_id}", response_model=JobStatus)
def get_job_status(job_id: str):
    if job_id not in _jobs:
        raise HTTPException(status_code=404, detail="Job not found")
    return _jobs[job_id]

# Client flow: POST /jobs -> {job_id, status: "pending"}
#              GET /jobs/{job_id} repeatedly until status == "complete"

```

Production note: `BackgroundTasks` runs in the same process — fine for light work, but for true durability (surviving restarts, retries, distributed workers) use a task queue like Celery, Arq, or a dedicated workflow engine, with the same polling/webhook API shape shown above.

13. File Uploads

Concept: FastAPI's `UploadFile` streams the file to a temp location on disk (avoiding loading huge files fully into memory) and exposes an async, file-like interface for reading and saving.

```
from fastapi import FastAPI, File, UploadFile, HTTPException
from typing import List
import shutil
import os

app = FastAPI()
UPLOAD_DIR = "/tmp/uploads"
os.makedirs(UPLOAD_DIR, exist_ok=True)

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    # Validate content type before touching disk
    if file.content_type not in ("image/png", "image/jpeg", "application/pdf"):
        raise HTTPException(status_code=400, detail="Unsupported file type")

    destination = os.path.join(UPLOAD_DIR, file.filename)

    # Stream in chunks rather than reading the whole file into memory at once
    with open(destination, "wb") as buffer:
        while chunk := await file.read(1024 * 1024): # 1MB chunks
            buffer.write(chunk)

    return {
        "filename": file.filename,
        "content_type": file.content_type,
        "size_bytes": os.path.getsize(destination),
    }

@app.post("/upload-multiple")
async def upload_multiple(files: List[UploadFile] = File(...)):
    saved = []
    for file in files:
        destination = os.path.join(UPLOAD_DIR, file.filename)
        with open(destination, "wb") as buffer:
            shutil.copyfileobj(file.file, buffer) # sync helper, fine for small files
        saved.append(file.filename)
    return {"uploaded": saved}

# --- Combining a file with regular form fields ---
from fastapi import Form

@app.post("/upload-with-metadata")
async def upload_with_metadata(
    file: UploadFile = File(...),
    description: str = Form(...),
    is_public: bool = Form(default=False),
):
    return {
        "filename": file.filename,
        "description": description,
        "is_public": is_public,
    }
```

Interview note: Prefer `UploadFile` over `bytes` for the file parameter — `UploadFile` is backed by a spooled temp file (memory only up to a size threshold, then disk), whereas a plain `bytes` parameter loads the *entire* file into memory, which doesn't scale for large uploads.

14. Authentication vs. Authorization

Concept: These are frequently confused. **Authentication** answers “*who are you?*” (verifying identity — login, tokens). **Authorization** answers “*what are you allowed to do?*” (permission checks, applied *after* authentication succeeds).

```
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from typing import Annotated
from datetime import datetime, timedelta
import jwt # PyJWT

app = FastAPI()
SECRET_KEY = "dev-secret-change-in-production"
ALGORITHM = "HS256"

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

# --- Fake user "database" with roles ---
_users_db = {
    "alice": {"username": "alice", "password": "secret123", "role": "admin"},
    "bob": {"username": "bob", "password": "secret456", "role": "viewer"},
}

def create_access_token(data: dict, expires_delta: timedelta = timedelta(minutes=30)):
    to_encode = data.copy()
    to_encode["exp"] = datetime.utcnow() + expires_delta
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

# === AUTHENTICATION: who are you? ===
@app.post("/token")
def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]):
    user = _users_db.get(form_data.username)
    if not user or user["password"] != form_data.password:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    token = create_access_token({"sub": user["username"], "role": user["role"]})
    return {"access_token": token, "token_type": "bearer"}

def get_current_user(token: Annotated[str, Depends(oauth2_scheme)]):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username = payload.get("sub")
        if username is None or username not in _users_db:
            raise HTTPException(status_code=401, detail="Invalid token")
        return {"username": username, "role": payload.get("role")}
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="Token expired")
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401, detail="Invalid token")

# === AUTHORIZATION: what are you allowed to do? ===
def require_role(required_role: str):
    """A dependency FACTORY — returns a dependency configured per-route."""
    def role_checker(user: Annotated[dict, Depends(get_current_user)]):
        if user["role"] != required_role:
            raise HTTPException(status_code=403, detail="Not enough permissions")
    return role_checker
```

```

        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail=f'Requires '{required_role}' role, you have '{user['role']}',
        )
    return user
return role_checker

@app.get("/profile")
def read_profile(user: Annotated[dict, Depends(get_current_user)]):
    # Authentication only – any logged-in user can access this
    return {"username": user["username"], "role": user["role"]}

@app.delete("/admin/users/{username}")
def delete_user(username: str, admin: Annotated[dict, Depends(require_role("admin"))]):
    # Authentication (via get_current_user, chained inside require_role)
    # PLUS Authorization (must specifically be "admin")
    return {"deleted": username, "by": admin["username"]}

```

The distinction in one line: `get_current_user` = authentication (401 if it fails — “I don’t know who you are”).
`require_role("admin")` = authorization (403 if it fails — “I know who you are, but you’re not allowed”).

Quick Reference Cheat Sheet

Advanced Python

Topic	One-line takeaway
<code>*args</code> / <code>**kwargs</code>	Collects extra positional/keyword args into a tuple/dict; essential for wrappers
Comprehensions	Declarative, fast container building; avoid nesting beyond 1 level
Generators/ <code>yield</code>	Lazy, memory-efficient iteration; pauses/resumes at each <code>yield</code>
<code>lambda</code>	Anonymous single-expression functions; only for trivial, throwaway logic
Decorators	Wrap a function to add behavior without changing its source; always use <code>functools.wraps</code>
Context managers	Guarantee setup/teardown via <code>__enter__</code> / <code>__exit__</code> or <code>@contextmanager</code>
Exceptions	Catch specific types; use <code>finally</code> for cleanup, chain with <code>raise ... from e</code>
<code>typing</code> / <code>Protocol</code>	Static hints + structural typing (duck typing the type-checker understands)
Dataclasses	Auto-generates <code>__init__</code> / <code>__repr__</code> / <code>__eq__</code> ; use <code>field(default_factory=...)</code> for mutables
<code>classmethod</code> / <code>staticmethod</code> / <code>property</code>	Alt constructors / namespaced utils / attribute-like computed values
Mutable defaults	Default args evaluate once at def-time — use <code>None</code> sentinel instead
Shallow vs. deep copy	Shallow shares nested objects; deep fully duplicates them
SOLID/DI	Depend on abstractions (Protocols/ABCs), not concrete classes — enables testing & swapping
<code>async</code> / <code>await</code>	Coroutines suspend at I/O boundaries; must be awaited or scheduled to run
<code>asyncio.gather</code> / Tasks	Run coroutines concurrently; <code>create_task</code> starts work immediately in background
Concurrency vs. parallelism	<code>asyncio</code> = one thread, interleaved (I/O-bound); <code>multiprocessing</code> = real parallel cores (CPU-bound)

Advanced FastAPI

Topic	One-line takeaway
Pydantic v2	Runtime-validating models; <code>.model_dump()</code> / <code>.model_dump_json()</code> replace v1's <code>.dict()</code> / <code>.json()</code>
Request/response models	Separate input/output schemas prevent leaking sensitive fields
Type hints	Drive automatic parsing: simple types → path/query, <code>BaseModel</code> → JSON body
Dependency injection	<code>Depends()</code> resolves, caches per-request, and supports <code>yield</code> for cleanup
DI for testing	<code>app.dependency_overrides[dep] = fake_dep</code> swaps real deps for test doubles
Async endpoints	<code>async def</code> for async-native I/O; plain <code>def</code> (thread pool) for blocking/CPU work
Blocking vs. non-blocking	Never call blocking code inside <code>async def</code> — it freezes the whole event loop
<code>asyncio.gather</code> in routes	Fetch independent upstream calls concurrently, not sequentially
Exception handling	<code>HTTPException</code> for expected errors; <code>@app.exception_handler()</code> for global JSON shape
Middleware	Wraps every request/response; executes in nested “onion” order
WebSockets	Persistent full-duplex connection for chat, live dashboards, token streaming
Long-running AI workflows	Stream (SSE), or background task + polling, for multi-second/minute jobs
File uploads	Prefer <code>UploadFile</code> (spooled temp file) over <code>bytes</code> (loads fully into memory)
AuthN vs. AuthZ	AuthN = “who are you?” (401); AuthZ = “what can you do?” (403, checked after AuthN)

End of guide.